Experiment 2

System Verilog Data Types and Looping statements

AIM:

To familiarize with the different data types and flow controls statements in System Verilog.

SOFTWARE/TOOLS USED:

Synopsys VCS

Questions:

Variables Declarations

- Declare variables to store the following types and initialize each with any value.
    - 2-state - 8 bit, 16 bit, 32 bit and 64 bit signed integer
    - 2-state 32-bit unsigned integer
- Declare a dynamic array 'data_da' of int data type.
- Declare queues 'data_q' and 'addr_q' of int data type
- Declare associative array 'data_mem' of int data type and indexed with bit[7:0]

Usage of Variables

- Allocate 10 memory locations for the dynamic array & initialize all the locations with some random values less than 25
    - Display the array
    - Call the array reduction method sum which returns the sum of all elements of the array and collect the return value to the variable result
    - Display the sum of elements, result
    - Sort the result and display the sorted array
- Queues
    - Use methods including size, insert, delete, push_front, pop_front on data_q and display content of data_q after execution of each method.
    - Generate 10 random addresses using repeat loop and push them in to addr_q and display the value of addr_q
- Associative array
    - Update the associate array with random data based on the address stored in addr_q (you may use pop to get the address).
    - Display the contents of the associate array using the 'foreach' loop
    - Display the first index of the array by using associative array method 'first'
    - Display the first element of the array
    - Display the last index of the array by using associative array method 'last'
    - Display the last element of the array
- User defined type
    - Create a user define type to store the pixel values of an image
    - Create 5 pixels and initialize them with values corresponding to red, violet and pink. You may use the following link to find out intensity values for different colors (https://a.atmos.washington.edu/~ovens/javascript/colorpicker.html)
    - Store the pixels in a packed array and display the values using foreach loops.

- Enumerated types
  - Write code for any state machine using enumerated type for state names and display the state names.
- Strings
  - Define and initialize a string and use any 3 string methods and display the results.

THEORY:

SystemVerilog is an extension of Verilog that includes additional data types and programming features which make coding and verification easier. It supports better data handling through arrays, queues, user-defined types, strings, and looping constructs. This experiment is aimed at understanding these features through simple examples and simulation.

1. SystemVerilog Data Types

SystemVerilog supports both 2-state and 4-state data types. In this experiment, 2-state integer data types such as byte, shortint, int, and longint are used. These data types store only 0 and 1 and are efficient for arithmetic and logical operations. A 32-bit unsigned integer is also used.

2. Dynamic Arrays

Dynamic arrays are arrays whose size can be decided during runtime. They are declared using empty brackets [] and memory is allocated using the new() keyword. Built-in methods like sum() and sort() are used to perform operations on the array. Dynamic arrays are useful when the amount of data is not fixed.

3. Queues

Queues are ordered collections of elements that allow insertion and deletion from both ends. They are commonly used to represent FIFO-type behavior. Operations such as push_front(), push_back(), pop_front(), insert(), delete(), and size() are used in this experiment to understand queue handling.

4. Associative Arrays

Associative arrays are indexed using arbitrary data types instead of continuous integers. They are useful for storing data at specific addresses. In this experiment, an associative array indexed by bit[7:0] is used, and the first() and last() methods help in accessing boundary elements.

5. User Defined Types (Structures)

User-defined data types are created using typedef struct. This allows related data items to be grouped together. A structure is used to represent pixel data using RGB values, and an array of such structures is used to store multiple pixels.

6. Enumerated Types

Enumerated types (enum) are used to define a set of named states. They improve code readability and are commonly used in FSM designs. In this experiment, enum methods like name() and next() are used to display and move through different states.

7. Strings

Strings in SystemVerilog are dynamic and provide built-in functions such as len(), toupper(), tolower(), and substr(). These functions make string handling easy and are useful for displaying messages in simulation.

8. Looping Statements

Looping constructs such as foreach, repeat, and while are used to initialize data, traverse arrays, queues, and associative arrays, and display their contents. These loops help reduce code complexity and improve readability.

SystemVerilog Code with Comments (#):

```
module datatypes;

  // ------------------------------------------------
  // Variable Declarations
  // ------------------------------------------------

  // 2-state signed integers (8-bit, 16-bit, 32-bit, 64-bit)
  byte      s8  = -8;
  shortint  s16 = -16;
  int       s32 = -32;
  longint   s64 = -64;

  // 2-state 32-bit unsigned integer
  int unsigned u32 = 32;

  // Dynamic array of int
  int data_da[];

  // Queues of int
  int data_q[$];
  int addr_q[$];

  // Associative array indexed by bit[7:0]
  int data_mem [bit [7:0]];

  int result;

  // User-defined type to store pixel values
  typedef struct packed {
    byte r;
    byte g;
    byte b;
  } pixel_t;

  pixel_t pixels[5];

  // Enumerated type (FSM)
  typedef enum {IDLE, READ, WRITE, DONE} state_t;
  state_t state;

  // String
  string msg = "Functional Verification Lab";

  bit [7:0] first_idx, last_idx;

  // ------------------------------------------------
```

```
// Initial Block
// ------------------------------------------------
initial begin

  $display("\n\n2-state :- 8 bit, 16 bit, 32 bit and 64 bit signed integer:");
  $display("s8=%0d s16=%0d s32=%0d s64=%0d u32=%0d",
           s8, s16, s32, s64, u32);

  // Dynamic array operations
  $display("\nDynamic array operations");
  data_da = new[10];

  foreach (data_da[i])
    data_da[i] = $urandom_range(0, 24);

  $display("Dynamic array: %p", data_da);

  // Sum
  result = data_da.sum();
  $display("Sum of elements = %0d", result);

  // Sort
  data_da.sort();
  $display("Sorted Dynamic Array: %p", data_da);

  // Queue operations
  $display("\nQueue operations");
  data_q.push_back(100);
  data_q.push_back(200);
  $display("Data Queue after push_back: %p", data_q);

  data_q.push_front(50);
  $display("Data Queue after push_front: %p", data_q);

  data_q.insert(1, 150);
  $display("Data Queue after insert at index 1: %p", data_q);

  data_q.delete(3);
  $display("Data Queue after delete index 3: %p", data_q);

  data_q.pop_front();
  $display("Data Queue after pop_front: %p", data_q);

  $display("Queue size = %0d", data_q.size());

  // Generate random addresses
  $display("\nGenerate random addresses and push to addr_q");
  repeat (10)
    addr_q.push_back($urandom_range(0, 255));

  $display("Address Queue: %p", addr_q);

  // Associative array operations
```

```
    $display("\nAssociative array operations");

    while (addr_q.size() > 0) begin

      data_mem[addr_q.pop_front()] =
$urandom_range(0, 255);

    end


    $display("Associative Array Contents:");

    foreach (data_mem[i])

      $display("Index=%0d Value=%0d", i,
data_mem[i]);


    // First and last index

    data_mem.first(first_idx);

    data_mem.last(last_idx);


    $display("First Index=%0d First
Element=%0d",

              first_idx, data_mem[first_idx]);

    $display("Last Index=%0d Last
Element=%0d",

              last_idx, data_mem[last_idx]);


    // Enumerated FSM

    $display("\nEnumerated FSM states:");

    state = IDLE;

    repeat (4) begin

      $display("%s", state.name());

      state = state.next();

    end

    // String methods


  $display("\nString methods");

    $display("String = %s", msg);

    $display("Length = %0d", msg.len());

    $display("Uppercase = %s",
msg.toupper());

    $display("Lowercase = %s",
msg.tolower());

    $display("Substring = %s",
msg.substr(0,10));


    // User-defined pixels

    $display("\nUser defined pixels:");

    pixels[0] = '{255, 0,   0};    // Red

    pixels[1] = '{138, 43, 226};  // Violet

    pixels[2] = '{255, 192, 203}; // Pink

    pixels[3] = '{0,   255, 0};   // Green

    pixels[4] = '{0,   0,   255}; // Blue


    $display("Pixel Values:");

    foreach (pixels[i])

      $display("Pixel[%0d] R=%0d G=%0d
B=%0d",

                i, pixels[i].r, pixels[i].g, pixels[i].b);


    $finish;

  end


endmodule
```

OUTPUT:

```
Activities  Terminal    Dec 30 04:20
c2user52@vlsi52:~/vld035_vcs
File  Edit  View  Search  Terminal  Help
d.
Please use '-no_save' simv switch to avoid re-execution or '-suppress=ASLR_DETECTED_INFO' to suppress this message.
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-1_Full64; Runtime version V-2023.12-1_Full64;  Dec 30 04:19 2025


2-state :- 8 bit, 16 bit, 32 bit and 64 bit signed integer:
s8=-8 s16=-16 s32=-32 s64=-64 u32=32

Dynamic array operations
Dynamic array: '{13, 10, 23, 20, 7, 1, 21, 3, 22, 7}
Sum of elements = 127
Sorted Dynamic Array: '{1, 3, 7, 7, 10, 13, 20, 21, 22, 23}

Queue operations
Data Queue after push_back(100) and push_back(200): '{100, 200}
Data Queue after push_front(50): '{50, 100, 200}
Data Queue after inserting 150 in [1]th position: '{50, 150, 100, 200}
Data Queue after deleting [3]th element: '{50, 150, 100}
Data Queue after pop_front(): '{150, 100}
Queue size = 2

Generate random addresses and push to addr_q
Address Queue: '{181, 250, 78, 21, 125, 114, 150, 49, 196, 170}

Associative array operations
Associative Array Contents:
Index=21 Value=244
Index=49 Value=44
Index=78 Value=79
Index=114 Value=136
Index=125 Value=23
Index=150 Value=241
Index=170 Value=5
Index=181 Value=196
Index=196 Value=206
Index=250 Value=207
First Index=21 First Element Value=244
Last Index=250 Last Element Value=207

Enumerated FSM states:
IDLE
READ
WRITE
```

```
Activities  Terminal    Dec 30 04:20
c2user52@vlsi52:~/vld035_vcs
File  Edit  View  Search  Terminal  Help

Associative array operations
Associative Array Contents:
Index=21 Value=244
Index=49 Value=44
Index=78 Value=79
Index=114 Value=136
Index=125 Value=23
Index=150 Value=241
Index=170 Value=5
Index=181 Value=196
Index=196 Value=206
Index=250 Value=207
First Index=21 First Element Value=244
Last Index=250 Last Element Value=207

Enumerated FSM states:
IDLE
READ
WRITE
DONE

String methods
String = Functional Verification Lab
Length = 27
Uppercase = FUNCTIONAL VERIFICATION LAB
Lowercase = functional verification lab
Substring = Functional

User defined pixels:
Pixel Values:
Pixel[0] R=-1 G=0 B=0
Pixel[1] R=-118 G=43 B=-30
Pixel[2] R=-1 G=-64 B=-53
Pixel[3] R=0 G=-1 B=0
Pixel[4] R=0 G=0 B=-1
$finish called from file "trial.sv", line 127.
$finish at simulation time                 0
           V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.100 seconds;       Data structure size:   0.0Mb
Tue Dec 30 04:19:43 2025
CPU time: .096 seconds to compile + .083 seconds to elab + .099 seconds to link + .121 seconds in simulation
[c2user52@vlsi52 ~/vld035_vcs]$
```

INFERENCE:

This experiment helps in understanding the practical use of different SystemVerilog data types and looping statements. The use of 2-state signed and unsigned variables demonstrates how numerical data can be represented efficiently. Dynamic arrays show how memory can be allocated during runtime and how built-in functions like sum and sort can be applied easily. Queue operations explain how ordered data can be managed using various insertion and deletion techniques, while associative arrays demonstrate sparse storage and indexed access using methods such as first, last, and foreach.

The implementation of user-defined structures for pixel data shows how related information can be grouped together using packed structures. Enumerated types make state representation clearer and easier to understand in FSM-based designs. String operations highlight the availability of built-in functions for handling text in testbenches. Overall, this experiment provides a good understanding of advanced data handling, user-defined data types, and control flow features in SystemVerilog, which are important for hardware design and verification.